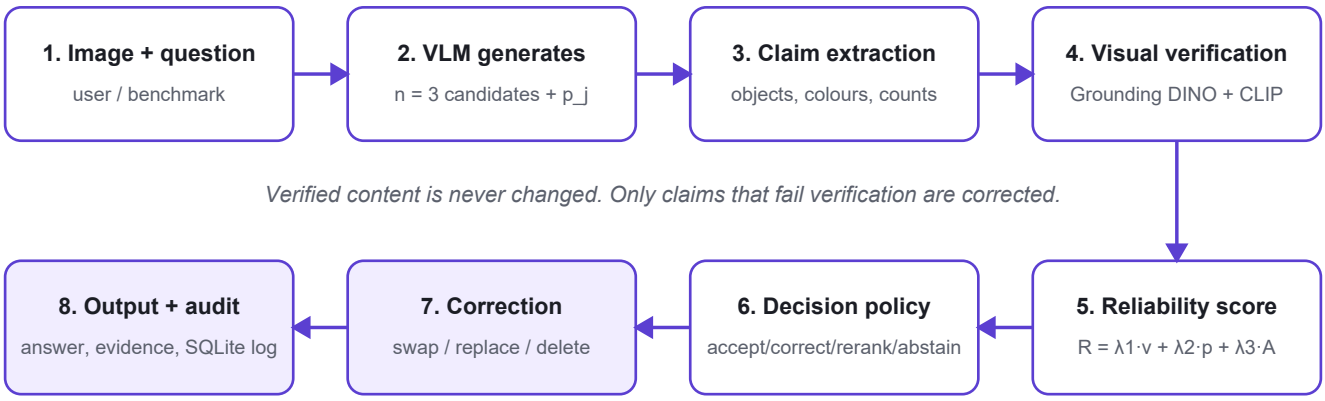
VLM Reliability Toolkit — Demo Script for Four Presenters

Aligned to the latest paper (rm_paper_yh.pdf). Every number below comes from the paper's tables or from a live run of the dashboard on 30 Oct 2026 with the paper's tuned thresholds.

#	Presenter	Topic	Paper	Live demo	Time
1	Prithvi	Problem, architecture, candidates and claims	I, III, IV-A–C	Scene A: live Gemini (8jkx)	4 min
2	Kavyansh	Visual verification and correction	IV-D, IV-F (edits), VI-D	Scene B: wrong caption (7nkn)	4 min
3	Harsh	Reliability score, decision policy, tuning	IV-E, IV-F, V-B	Scene C: re-ranking (dqp7)	4 min
4	Yash	Benchmarks, ablations, limitations	V, VI, VII, VIII	Evaluation + Experiments pages	4 min

Total about 16 minutes, then 4–5 minutes of questions. Each person explains one stage of the pipeline and drives one thing on screen, so everyone gets an equal share.



Stages 1–3: Prithvi · stage 4 and 7: Kavyansh · stages 5–6: Harsh · benchmarks: Yash

PART 1 · PRITHVI about 4 minutes

The problem, the architecture, and a live model

Paper sections: I Introduction, III Problem Formulation, IV-A Architecture, IV-B Candidates, IV-C Claim Extraction

Open with the problem (45 s)

SAY — Prithvi: Vision-language models like LLaVA write fluent descriptions, but they make two kinds of mistakes. They mention objects that are not in the image, which is object hallucination. And they attach an attribute to the wrong object: the image shows a red cup on a blue table, and the model says a blue cup on a red table. That is an attribute-binding error. Checking that an object exists does not tell you its colour is right, so we need both checks. Our question was: can we detect and correct both, in one pipeline, without training anything, and without touching the parts of the answer that are correct?

Architecture (60 s) — show Fig. 1 or docs/images/analysis-pipeline.png

SAY — Prithvi: The toolkit is a layer that runs after any VLM. It treats the model as a black box, or as a white box when token probabilities are available. There are seven components: the VLM engine, an orchestrator that asks for n candidate answers, a diagnostic engine that extracts and verifies claims, the reliability engine, the decision and correction layer, the benchmark module, and this dashboard. One rule runs through all of it: no language model judges another language model. Every decision comes from detector boxes and image crops.

Live demo — Scene A (90 s)

- Analyze page. Clear the override box. Upload **Gemini_Generated_Image_8jkx…png** (busy city street).
- Question: *Describe the objects and their colors.* Provider Gemini, 3 candidates. Click **Run reliability analysis** (about 10–20 s). Talk through the next box while it runs.
- When it returns, open the **Claims** page and the claim graph.

SAY — Prithvi: We ask for three candidates. Candidate 1 is greedy decoding: that is the baseline we report against. Candidates 2 and 3 are sampled at temperature 0.7, seed 42. For API models the three calls run in parallel; for local models we also get the token probability, p_j. Each answer is then split into atomic claims by a deterministic, rule-based extractor — again, no LLM. There are four claim types: an object exists, an object has a colour or material, a count, and a spatial relation. The vocabulary is the 80 COCO classes with the CHAIR synonyms, 11 colours and 8 materials. Every claim keeps its character span, so later we can edit exactly those words and nothing else.

- Expected on screen: cars, yellow taxis, buildings and people are ACCEPTed; *blue sedans* and *green trees* are flagged as uncertain and left unchanged (decision ABSTAIN, R ≈ 0.56).
- **Fallback (no Wi-Fi or Gemini error):** open the Experiments page and reopen the earlier Gemini run of 8jkx from the audit log. Say: "every run is stored, so here is the one we ran this morning."

Hand-off line: *We now have claims. Kavyansh will show how we check each one against the image.*

PART 2 · KAVYANSH about 4 minutes

Visual verification and evidence-gated correction

Paper sections: III Eq. 1, IV-D Eq. 3 and 4, IV-F correction operations, VI-D example

Explain the verifiers (75 s)

SAY — Kavyansh: Every claim gets a score v_i between 0 and 1 and a status: supported, contradicted, unsupported or ambiguous. For existence we ask Grounding DINO, an open-vocabulary detector, to find the object. The score is the confidence of its best box, $s_{obj} = \max d_b$ — that is Equation 3. At 0.30 or above the object is supported. For attributes, we crop each of the object's boxes and ask CLIP: is this a photo of a red shirt, a blue shirt, and so on. The binding score is detection confidence times the crop probability, Equation 4. The key point is that we only look inside the shirt's own box. A blue sky elsewhere in the picture can never support "blue shirt". Counts use non-maximum suppression, and relations use box geometry.

Live demo — Scene B (90 s)

- Upload **Gemini_Generated_Image_7nkn…png** (a man in a red T-shirt on a street).
- Open **Controlled candidate override** and paste: *A man in a blue shirt walks down the street holding a surfboard.* Run (about 1–5 s).
- **Evidence canvas:** boxes for the person and the shirt; no box for a surfboard.
- **Claims table**, point at each row:

Claim	Status	v	Action
person exists	SUPPORTED	0.88	ACCEPT
shirt exists	SUPPORTED	0.56	ACCEPT
shirt is blue	CONTRADICTED (crop is red)	0.04	CORRECT — replace blue → red
surfboard exists	UNSUPPORTED (no detection)	0.12	CORRECT — delete clause

- **Correction diff:** *A man in a red shirt walks down the street.*

Explain the correction rule (45 s)

SAY — Kavyansh: We only change text when the image gives positive evidence for the replacement. The shirt crop clearly reads red, so blue becomes red. The surfboard has no detection, so its clause is deleted. Everything that was verified is left word for word, because we edit character spans, not regenerate the sentence. There are four edits: replace, swap — for example "blue shirt and red jeans" becomes "red shirt and blue jeans" when each region shows the other colour, which is the example in Section VI-D — recount, and delete. If the evidence is weak, we do not guess; the claim is flagged.

Hand-off line: *Checking each claim is half the job. Harsh will show how we score the whole answer and decide what to do.*

PART 3 · HARSH about 4 minutes

Reliability scoring and the decision policy

Paper sections: IV-E Eq. 5, IV-F Table II and re-rank rule, V-B threshold tuning

Explain the score (60 s)

SAY — Harsh: Each candidate gets one reliability score, R, Equation 5. It combines three things: the mean verification score of its claims with weight 0.6, the model's own token confidence with weight 0.2, and agreement — how many of its claims the other candidates also make — with weight 0.2. If a term is not available, for example token confidence from an API model, we do not invent it; its weight is shared out over the other terms. R is used only for the whole answer. Each individual claim is judged on its own score.

Explain the policy (45 s) — show Table II

SAY — Harsh: Per claim: at or above the high threshold we accept, at or below the low threshold we correct if there is evidence for a fix, and in between we flag it as uncertain. Per answer: if the greedy answer has R below 0.5 and another candidate reaches 0.5, we switch to that candidate. That is re-ranking. The thresholds are not hand picked: we tuned them on 50 held-out POPE images that are never reported, which gave 0.15 and 0.20 for objects at an F1 of 0.848.

Live demo — Scene C (90 s)

- Upload **Gemini_Generated_Image_dqp7…png** (dog, cat and an old car in a park). Paste three candidates separated by **|||**:
  A golden retriever dog and a white cat sit on the grass near a green car and a red bicycle. ||| *A dog and a cat sit near a car.* ||| *A brown dog sits on a bench next to two cats and a blue umbrella.*
- **Decision panel:** RERANK — *Top candidate R=0.49 < 0.50; candidate-2 has R=0.86.*
- **Candidates page:** chart and cards (R 87% / 49% / 46%).
- **Reliability panel:** claims dog 0.79, cat 0.88, car 0.81, near 0.81 · agreement 100% · p_i n/a.

SAY — Harsh: The first answer invents a bicycle and gets details wrong, so its R is 0.49, just under the line. The second is simpler but fully verified. Its four claims average 0.8225, agreement is 1, and there is no token probability, so R = (0.6 × 0.8225 + 0.2 × 1) / 0.8 = 0.867. That is exactly the worked example in Section IV-E of our paper. Also notice Kavyansh's scene: R was only 0.40 there, yet the answer was corrected, not rejected, because a low R alone does not trigger abstention when we have evidence-backed corrections.

Hand-off line: *That is three images. Yash will show whether it holds up over thousands.*

PART 4 · YASH about 4 minutes

Benchmarks, ablations and limitations

Paper sections: V Setup, VI Results (Tables IV and V, Fig. 2), VII Limitations, VIII Conclusion

Setup (30 s)

SAY — Yash: We ran LLaVA-1.5-7B on Kaggle with two T4 GPUs. Every detector and CLIP output is cached by the image's SHA-256 hash, so the full POPE and CHAIR runs can be re-scored on a laptop with no GPU in about 20 seconds.

Results — open the Evaluation page (90 s)

CHAIR, 500 COCO images	$CHAIR_S \downarrow$	$CHAIR_I \downarrow$	Recall ↑	Words	Latency
Base VLM	63.8	29.6	76.2	86.4	1×
Full toolkit	**57.2**	**25.5**	75.4	83.9	2.67×
n = 1	59.6	27.2	75.8	84.4	1.23×
No reliability scoring	59.6	27.2	75.8	84.4	2.67×
No attribute binding	57.6	26.0	75.5	83.9	2.67×

SAY — Yash: On CHAIR the toolkit cuts captions with a hallucinated object from 63.8 to 57.2 percent, and hallucinated object mentions from 29.6 to 25.5 percent. Recall drops by only 0.8 points and captions lose 2.5 words, so we are not just deleting content. The cost is 2.67× the latency.

Ablations (45 s) — Fig. 2, then POPE (30 s)

SAY — Yash: A single candidate, n = 1, already gets most of the gain at 1.23× latency; re-ranking over three adds 2.4 and 1.7 points. Without reliability scoring the extra candidates are never used, so it equals n = 1. Removing the attribute probe barely matters here, because CHAIR has no attribute ground truth.

SAY — Yash: On POPE the effect is small, and we say so. The toolkit can only turn a yes into a no, and LLaVA already says yes to only about 40 percent of a balanced benchmark; its main error is missed objects. Precision still rises on every split: 96.9 → 97.8 random, 94.7 → 95.1 popular, 89.6 → 90.3 adversarial, with accuracy within 0.1 points.

Audit log and close (45 s) — open the Experiments page

SAY — Yash: Every live run today is stored here with its image hash, candidates, scores and decision. Limitations: we are only as good as the detector, colour is sensitive to lighting, attribute correction is not yet measured by a benchmark, and all results are one model, one seed. Next we will evaluate attribute binding on Winoground and SugarCrepe, add significance tests and more generators, and reduce latency. Thank you — we are happy to take questions.

Questions — who answers

Question	Owner	Answer in one or two lines
How is this different from Woodpecker?	Prithvi	Woodpecker also corrects after generation but uses language models in some stages. Ours is fully deterministic, models attribute binding explicitly, and can abstain.
What if the detector misses a small object?	Kavyansh	Then we may delete a real object. That is why recall is reported (−0.8 points) and why a low score alone is not treated as proof of absence.
Why did it not correct the blue sedans?	Kavyansh	Its score was about 0.15, right at the low threshold, so it fell in the uncertain band. The street has cars of many colours, and without clear evidence for one substitute we flag instead of editing.
Why these weights 0.6 / 0.2 / 0.2?	Harsh	An ECE search on POPE puts all weight on token confidence, because POPE answers are one yes/no token. Applied to captions that ignores the image: $CHAIR_S$ 59.4 instead of 57.2. So we keep the visual-dominant weights.
Is R a calibrated probability?	Harsh	No. ECE of R is about 0.2–0.27 on POPE. It ranks candidates well, but needs human-annotated calibration before it can be used as a confidence.
Is 6.6 points significant?	Yash	We have one run and one seed; paired resampling and more generators are planned. We state this as a limitation.
Why is it 2.67× slower?	Yash	Three generations plus detection and CLIP per claim. n = 1 gives most of the gain at 1.23×.

Paper vs implementation — checked 30 Oct 2026

Matches the code

- Generation: n = 3, temperature 0.7, seed 42, 128 tokens; greedy candidate 1; GENERATION_FAILED keeps its slot.
- Vocabulary: 11 colours and 8 materials exactly as listed; NMS IoU 0.5; the near / next-to rule (L1 distance < 2 × larger box's width plus height); the four edits including recount.
- Eq. 3–5, the floor 0.10, grounding threshold 0.30, P ≥ 0.5, λ = (0.6, 0.2, 0.2), τ_R = 0.5, and the re-rank / abstain rules in core/policy.py.
- Every number in the abstract and Tables IV and V matches results/RESULTS.md. The IV-E worked example (0.867) is reproduced live by Scene C.

Fixed today

- The dashboard folder had no results/tuned.yaml, so live runs used object thresholds 0.30 / 0.45 instead of the paper's tuned 0.15 / 0.20. Copied it in; all three scenes were re-run and give the results in this script.

Worth fixing in the paper text

- IV-A says the dashboard is "for human review". The Review page was removed; say "for inspection and audit".
- Table I caption: "Claim Types Handled by the Extract" → "…by the Extractor".
- Limitations: "(vii) Object thresholds are tuned on POPE…" is merged into the previous bullet; give it its own.
- Abstract: "reranking" is split across a line as "re-ranking"; equation numbers sit against the formulas ("$d_b(3)$") — add spacing.
- V-D says 78 tests. That is true on the new-ui branch; after merging fix/reliability-weights it is 81.

- V-D: "about 3× faster, scores within ±0.03" for 480 × 800 Grounding DINO — confirm from your measurement notes; it was not re-measured today.

Small code inconsistency

- core/policy.py still says "escalated for human review" when no candidate is reliable. None of the demo scenes hit that branch, so it will not appear on screen.

Before the demo

- Start: powershell -ExecutionPolicy Bypass -File scripts\dev.ps1; check http://127.0.0.1:8010/health and http://127.0.0.1:3000/analyze.
- Run scenes A, B and C once so they return from the cache; keep the two override captions in a notes file.
- Evaluation page shows the POPE and CHAIR tables; close other heavy apps (verification runs on CPU).
- If Gemini fails: Prithvi uses the Experiments-page fallback; scenes B and C need no API.